01. Collaborative Pi Durable



Target time: 15 seconds

Speaking points

- I extended Pi Durable itself so several people can instruct shared coding agents in the same project folder. The central engineering work is deciding what gets saved, how shared requests are ordered, and what recovery can safely repeat.
- This is a take-home prototype built with AI assistance and reused upstream code. I will distinguish implemented behavior, measured evidence, and planned extensions.

Suggested demonstration: No interaction yet. Keep two authenticated browser sessions ready on the future GCP host and use a disposable presentation thread/store.

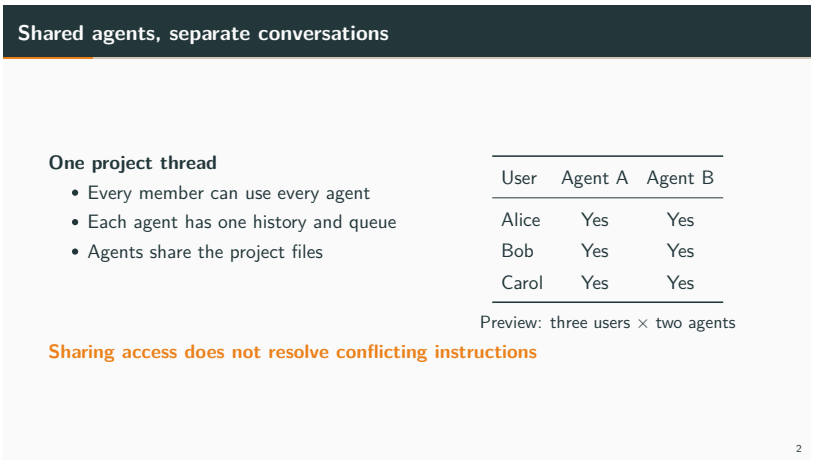
Fallback: Use the included local screenshot and controlled file trace if the model or hosted app is unavailable.

Boundary to state: Initial GCP VM/service startup and HTTP 200 were user-confirmed. The new features have local deterministic evidence; their hosted behavior, public HTTPS and complete cloud acceptance are not verified.

References: Wiki: the short project pitch

Source: PROJECT_INSTRUCTIONS.txt

02. Shared agents, separate conversations



Target time: 25 seconds

Speaking points

- M:N means users share agent identities, not that every person gets a private copy of each agent. Alice and Bob can both address A and B; A has one shared conversation and queue, and B has a separate conversation.
- Both agents see the same workspace files. The three-by-two table illustrates the relationship model and matches the deterministic preview; the normal real-provider example has two users and two agents.
- The system can record Alice asking for React and Bob asking for Vue. Ordering and attribution do not decide which product requirement should win.

Suggested demonstration: On the future GCP app, show the member list, both agent panels, and the access/coordination section. Use Alice and Bob for the live four pairings; add Carol only in a separately configured fresh store if you want six pairings.

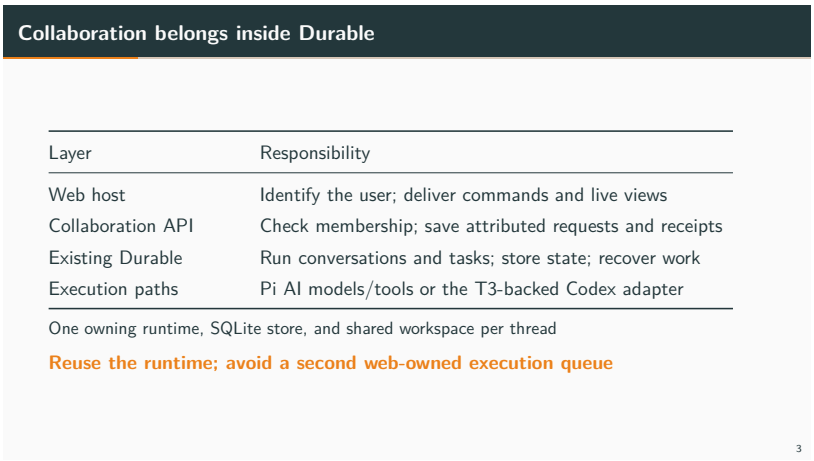
Fallback: Point to the access matrix and explain why two pages observe the same agent history.

Boundary to state: Membership is fixed for a saved store. The matrix shows access, not online presence or per-agent privacy; agents do not automatically share every conversation message.

References: Wiki: many to many sharing is a relationship model

Wiki: why do two agents share files but keep separate conversation histories

03. Collaboration belongs inside Durable



Target time: 45 seconds

Speaking points

- The host identifies Alice from her login token. The collaboration package checks membership and saves her attributed request; existing Durable conversations, inboxes, tasks, and storage run the work.
- The key design decision was avoiding a second queue and database in the web server. Otherwise the server could accept a request and crash before handing it to Durable. Saving the receipt and work together removes that particular gap.
- Durable already owns its model/tool loop and recovery; this is not an exception wrapper around coding-agent. The separate coding-agent CLI/SDK also has execution and persistence, but our demo does not run it. Chord supplies selected standalone utilities, not its full application host. This upstream overlap predates our fork.

Suggested demonstration: Keep this architectural explanation on the slide. Optionally show that Alice's and Bob's browser pages use the same host; a standalone package example can exercise collaboration without the web server.

Fallback: Trace Alice -> authenticated host -> collaboration package -> Durable -> native tools or Codex -> shared workspace.

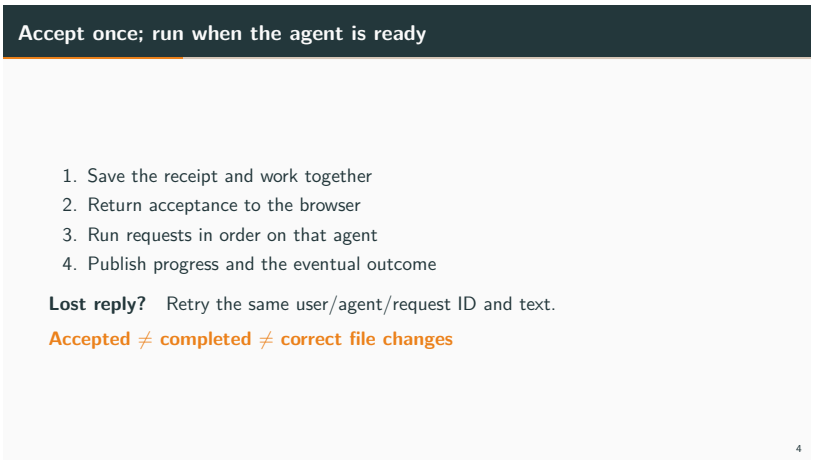
Boundary to state: The current model is one exclusive owner per thread store/workspace. The browser and read-only file viewer do not own the agent execution queue.

References: Wiki: why the feature lives inside durable

Wiki: what was added and what was reused

Wiki: thread conversation lane session and harness

04. Accept once; run when the agent is ready



Target time: 35 seconds

Speaking points

- Acceptance is a short saved-state decision; the model can take much longer. The HTTP 202 reply means the request was accepted, not that the requested code is correct. Work can begin after the save before that reply reaches the browser; the slide lists conceptual stages, not a promise to wait for network delivery.
- If the response is lost after saving, Alice retries the same thread/user/agent/request ID and text to find the original receipt. A new ID is new work; the same identity with changed text is rejected.
- The demo runs follow-ups one at a time on each agent. Independent agent lanes can overlap, while database commits take turns separately; neither rule protects all shared-file changes.

Suggested demonstration: If the first live instruction is still active, submit Bob's prepared follow-up to the same agent and point to queued status. Do not rely on typing speed. To show an actual ambiguous-response retry, use a separately rehearsed request harness preserving the exact original ID; there is no current GUI fault-injection button.

Fallback: Use the click-to-result sequence diagram in the linked wiki. A stable-ID retry test is more reliable than manually interrupting a fast live request.

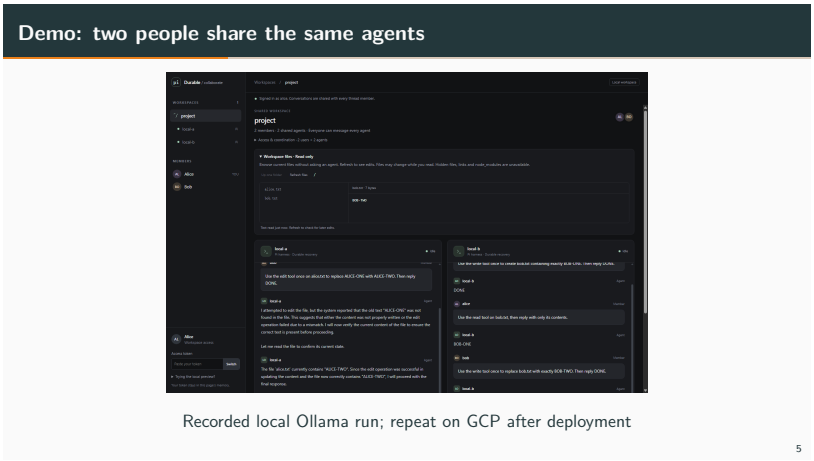
Boundary to state: Do not equate one accepted command with one model request or one file effect. Browser reload/relogin can lose its unsent or ambiguous local retry draft.

References: Wiki: a command from click to result

Wiki: retry identity and idempotent admission

Wiki: idempotency deduplication and exactly once claims

05. Demo: two people share the same agents



Target time: 60 seconds

Speaking points

- I will show shared observations and actual file bytes. A model's DONE message is not my test of success.
- The screenshot is from the recorded local Ollama run, not from GCP. It also shows a model describing an edit mismatch despite correct final bytes, which is why independent file inspection matters.
- Use the configured agent IDs shown in the app; A and B are explanatory names. Local Ollama can process inference serially even though agent histories and runtime lanes are independent.

Suggested demonstration: After GCP readiness: open two browser profiles and privately authenticate as Alice and Bob. Alice -> A: 'Use the write tool once to create alice.txt containing exactly ALICE-ONE. Then reply DONE.' Bob -> A: 'Use the edit tool once on alice.txt to replace ALICE-ONE with ALICE-TWO. Then reply DONE.' Bob -> B: 'Use the write tool once to create bob.txt containing exactly BOB-ONE. Then reply DONE.' Alice -> B: 'Use the read tool on bob.txt, then reply with only its contents.' Inspect ALICE-TWO and BOB-ONE in Workspace files, using manual Refresh. Reconnect/reload Alice and sign in again if needed; show saved history without resubmitting. Paste prompts prepared in advance.

Fallback: Use the screenshot and the recorded local browser evidence. If A finishes before Bob submits, explain that no wait was needed; do not claim that run demonstrated queueing. Use controlled tests to establish the queue rule.

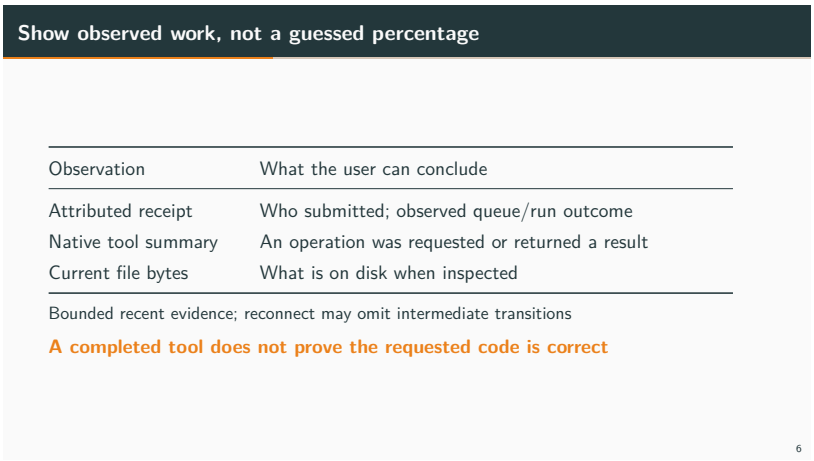
Boundary to state: Use a fresh disposable presentation store and rehearse these filenames. Do not overwrite a live team's files or claim that a screenshot establishes cloud deployment. Refresh is manual and the file view is not an atomic snapshot.

References: Wiki: what the current gui can prove

Wiki: the file browser is inspection not version control

Source: apps / collab demo / scripts / results / 2026 11 08 ollama browser.json

06. Show observed work, not a guessed percentage



Target time: 35 seconds

Speaking points

- The feed connects visible work to saved records: an attributed request can be queued or running, and native agent entries can contain tool requests and results. A completed tool is still not proof that the coding requirement is correct.
- The page keeps 64 distinct observations. Snapshots include eight recent receipts and up to 24 native tool records from twenty entries. Reconnect restores bounded current evidence, not all missed transitions or a global audit history.
- Tool summaries stay agent-scoped when assigning them to one user request would be a guess. They omit raw arguments/results. External harnesses retain their existing progress text rather than claiming structured native tool evidence.

Suggested demonstration: Expand Activity while the shared-agent instructions run. Point to the user on a receipt, then to a native read/write tool summary and the actual workspace file. If events have already completed, explain that these are saved observations; there is no fabricated percent-complete indicator.

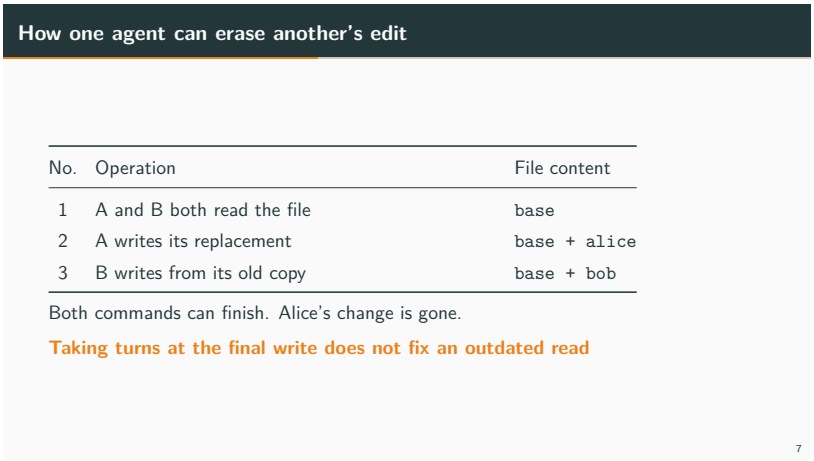
Fallback: Trace Alice sends -> request observed -> native tool requested/result -> inspect file. Use the recorded local GUI if the provider is slow.

Boundary to state: An observed request/tool transition is not a complete audit log, trustworthy line-by-line authorship or semantic coding success.

References: Wiki: what the activity feed means

Wiki: can the activity feed tell me who changed a particular line

07. How one agent can erase another's edit



Target time: 25 seconds

Speaking points

- Both agents can read the same old base before either writes. A saves base plus Alice, and B later saves base plus Bob from its stale copy, erasing Alice's change.
- The final writes happened in order, so serializing individual writes is insufficient. The gap is read, model planning, then write; shell commands and external Codex can also bypass Pi's individual mutation gate.
- This is a controlled lost-update example. Unrestricted real writes may also fail or leave partial content, so last-writer-wins is not a universal filesystem promise.

Suggested demonstration: Explain the unsafe trace, then switch to the implemented controlled conflict panel on slide 9. It lets two users prepare before either publishes; no typing race or model timing is needed.

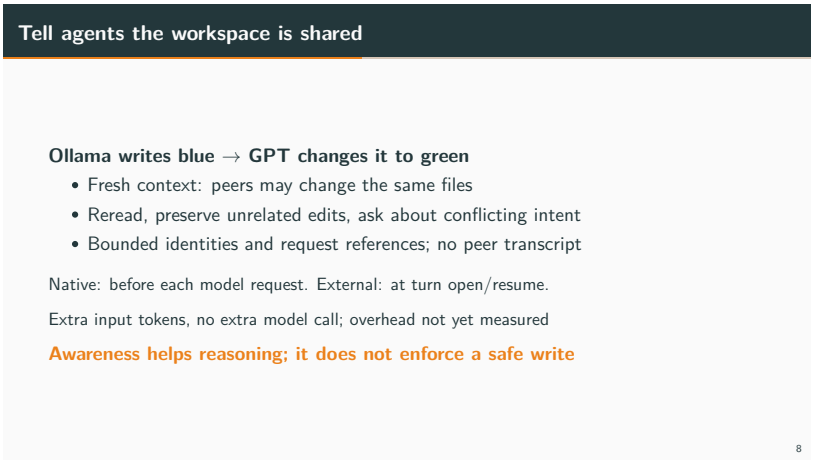
Fallback: The table is the entire causal trace. Ask the audience what B's remembered file contains after A writes.

Boundary to state: Ordinary model tools still permit this risk. Same-agent queues are not cross-agent conflict resolution.

References: Wiki: the lost update example to remember

Wiki: javascript runs on one event loop so how can two agents still lose a file edit

08. Tell agents the workspace is shared



Target time: 45 seconds

Speaking points

- Ollama writes blue; GPT later changes it to green. Ollama's separate history only explains blue, so without shared-workspace context it might interpret green as its own failed edit and restore blue.
- Before each native model request, the runtime refreshes a short policy, bounded participant IDs and active/recent request references from saved state. It tells agents to reread, preserve unrelated edits and ask if intent conflicts. It does not send peer conversation text or claim to know who changed each file.
- External harnesses receive context at turn open/resume, not continuously. There is no extra model call, but extra input tokens and local context updates have unmeasured cost/cache effects. Tests verify the supplied prompts and actual peer-file trace; useful live model compliance still needs checking.
- Awareness and rereading address stale assumptions. Another writer can act between read and write, so enforced managed publication is a separate layer. Neither this context nor the activity feed is a verified file-author log.

Suggested demonstration: After rehearsing with existing OpenAI/Ollama access, use a disposable file: Ollama writes blue, GPT changes it to green, then ask Ollama to reread and describe its current contents without restoring the old version. Independently inspect the file. Treat any unrehearsed model response as an observation, not a guaranteed result.

Fallback: Explain the blue -> green -> reread trace. The deterministic context tests capture the next native prompt and preserve existing-store configuration without making provider calls.

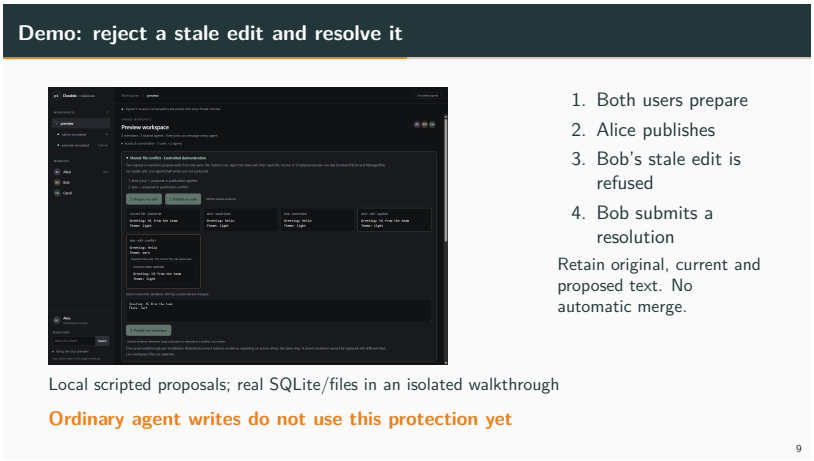
Boundary to state: Prompt guidance is not a lock, proof of authorship or a guarantee against lost updates. No live interpretation or token-overhead result is claimed.

References: Wiki: llm wiki / 10 collaboration awareness.md

Wiki: does rereading before writing guarantee that an edit is current

Source: packages / durable / src / collaboration / context.ts

09. Demo: reject a stale edit and resolve it



Target time: 100 seconds

Speaking points

- Each user selects a native model and enters a natural-language edit prompt. Generate saves that user's base and admits a proposal-only conversation without filesystem tools; only explicit Publish can change the file through ManagedFile. Both users must prepare before publication.
- Alice publishes first and Bob's old-base proposal is refused. Bob chooses explicit resolution text. Reset starts a fresh saved round for both users, restores the neutral file/default prompts and closes previous generation. Old-round actions are rejected; up to 64 rounds are retained without deleting live data.
- The screenshot remains historical local scripted-browser evidence. The current live panel generates with configured native providers; preview stays scripted. A separate real Ollama check generated both defaults, left the file unchanged during generation, then verified stale refusal and reset. Ordinary coding tools, shell and Codex remain outside this protocol.

Suggested demonstration: On the updated host, open the conflict panel in Alice and Bob sessions. Choose native models and edit My edit instruction, or retain the defaults. Both click 1. Generate my proposal and inspect the results. The initial file is Welcome/neutral so both defaults change it. Alice publishes, then Bob publishes; inspect the refused stale edit and saved versions. Bob submits an explicit resolution. Click Reset round for both users; show fresh defaults and unchanged live coding history. In preview the first button is Prepare sample proposal and outputs are fixed, not prompt interpretation.

Fallback: Use the included local conflict screenshot or run example 41 in a separate terminal: cd /opt/pi/packages/durable; node --conditions=source --experimental-strip-types test/examples/41-managed-file-conflict.ts. The example needs built packages but no server or model key.

Boundary to state: The live model generates text in an isolated tool-free conversation, not the normal coding lane. Provider usage applies. An unchanged or identical output may yield no conflict; inspect it. Reset does not bypass an uncertain managed publication or delete archives. No automatic merge, multi-file transaction or ordinary shell protection is claimed.

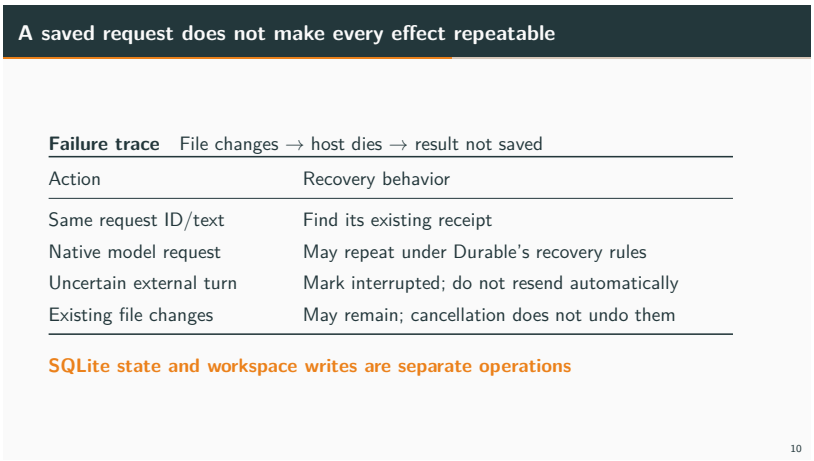
References: Wiki: how the scenario panel demonstrates a conflict

Wiki: managed publication and optimistic concurrency control

Source: apps / collab demo / src / conflict demo.ts

Source: apps / collab demo / scripts / results / 2026 11 10 conflict rounds ollama.json

10. A saved request does not make every effect repeatable



Target time: 55 seconds

Speaking points

- Suppose a tool appends hello and the host dies before saving success. A request receipt tells us the instruction was accepted, not whether the append happened; blindly repeating it could append twice.
- Native generation retains Durable's recovery rules, including possible repeated model requests and declared unsafe-tool no-replay. The external path saves a dispatch marker before attempting Codex and reports an uncertain attempt interrupted instead of automatically resending.
- The trade-off is that a crash after the marker but before send can interrupt work that never started. Cancellation does not undo existing file changes, and a stopped parent process does not prove every child writer has stopped.

Suggested demonstration: A safe hosted GUI demonstration is reconnect followed by a rehearsed idle service stop/start on a disposable store, then restored files/history. Keep active crash injection and external-effect tests separate in controlled fixtures; never kill the shared live host for an unrehearsed presentation.

Fallback: Walk through the append/crash diagram in the glossary. State separately what happens to the receipt, model request, and file effect.

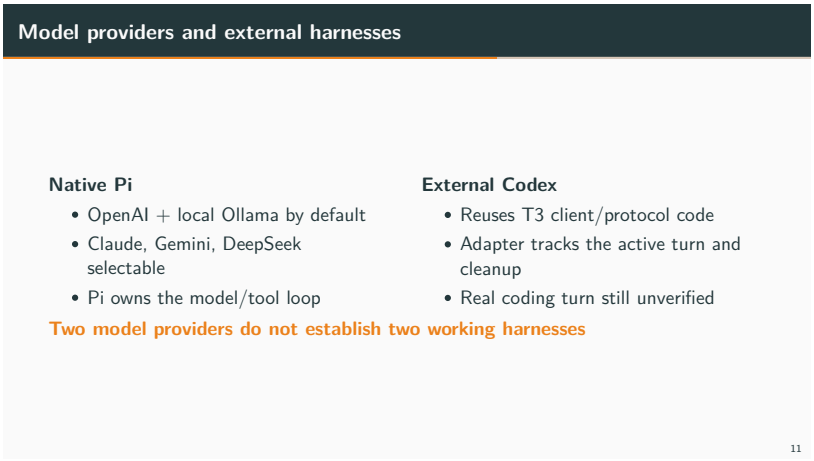
Boundary to state: An idle restart is not active-shell takeover or power-loss proof. Storage errors can happen after commit; the whole failed Harness must be stopped before reopen. Windows Bash shutdown investigation remains deferred.

References: Wiki: llm wiki / 03 durability and recovery.md

Wiki: side effects checkpoints and dispatch markers

Wiki: llm wiki / interview questions / 04 failures and recovery.md

11. Model providers and external harnesses



Target time: 30 seconds

Speaking points

- A provider supplies model output; a harness owns the surrounding conversation and tool loop. OpenAI plus Ollama uses two providers through Pi, whereas Codex is an external harness with its own loop.
- The Codex adapter actually executes vendored T3 typed client/protocol code, with replay support in tests and preserved provenance/licenses. It does not import T3's whole application, credential system, or UI.
- Scripted subprocess tests caught wrong-turn correlation and interrupt-delivery/cleanup races. Real CLI initialization works, but no recorded authenticated real Codex coding turn or continuation exists. Claude, Gemini, and DeepSeek have selectable provider paths; their live keys were not supplied for multi-key testing.

Suggested demonstration: On the future GCP host, show the configured OpenAI/Ollama agent names and use their existing successful instructions. Demonstrate a Codex lane only after separate service-user authentication and a real turn/continuation check; use a fresh store for changed agent configuration.

Fallback: Use the provider/harness diagram and the T3 provenance file instead of pretending a native OpenAI turn is Codex evidence.

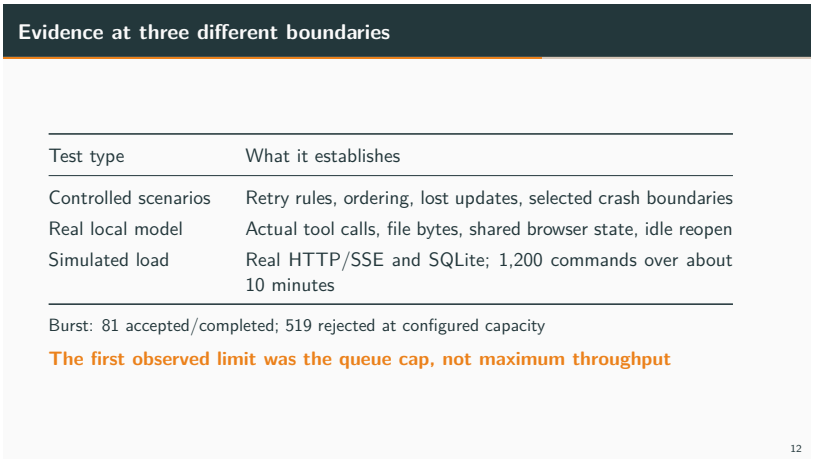
Boundary to state: Unsupported interactive Codex approvals/questions fail visibly; shared approval arbitration is not implemented. OpenAI API credentials and Codex authentication are separate prerequisites.

References: Wiki: exactly what t3 contributed

Wiki: provider model agent harness and adapter

Source: packages / durable / src / adapters / codex / vendor / UPSTREAM.md

12. Evidence at three different boundaries



Target time: 45 seconds

Speaking points

- Controlled tests force races and selected crash points; real local-model checks establish actual tool use and file bytes; load checks exercise real transport/storage with simulated work. These are different claims, and no category replaces the others.
- The roughly ten-minute soak completed 1,200 commands at an offered two per second across ten lanes. The burst accepted/completed 81 and rejected 519 at configured capacity, so the first limit observed was the queue cap, not a database throughput ceiling.
- Across baseline, burst, and soak, all 1,301 accepted logical commands had one recorded start; 30 retry probes added no starts. These are observations for simulated execution, not a universal exactly-once or real-model throughput guarantee.
- The new controlled browser check verifies actual managed-file outcomes in two identities. Context tests capture fresh metadata around peer edits and SQLite reopen; we have not yet measured live model interpretation or prompt overhead.
- A separate real Ollama check generated both default file proposals without tools, verified unchanged bytes before publication, then stale refusal and reset. It used disposable SQLite and is not an OpenAI or GCP check.

Suggested demonstration: Show the portable load report from the repository or a saved report in the SSH terminal. The current web app has no performance dashboard. Running the workload in another disposable owner can be rehearsed separately; avoid load against the presentation's live data.

Fallback: Use the slide's measured counts and explicitly describe simulated execution over real SQLite/HTTP/SSE.

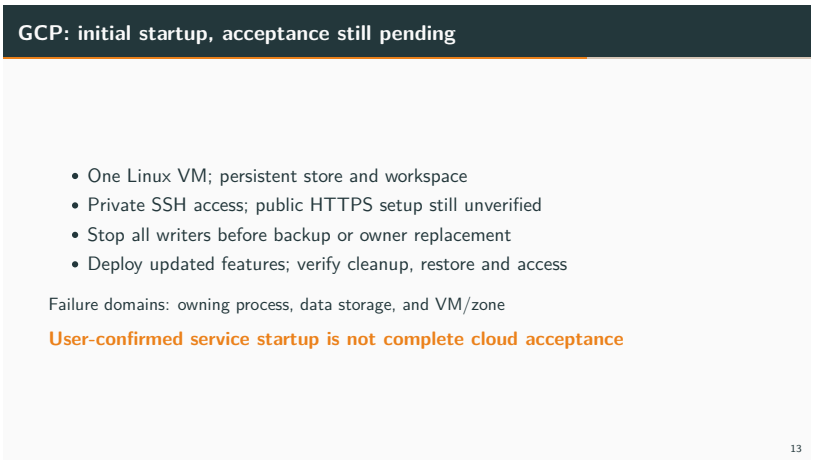
Boundary to state: Latest recorded checks: 48 demo tests across thirteen files and 68 focused Durable tests across seven files, plus root/app checks and build. These are selected suites, not all repo tests. Scripted browser and captured prompt checks do not prove live model compliance. Historical load p95/RSS are observations, not bounds.

References: Wiki: the load result worth remembering

Wiki: what the measurements cannot prove

Source: apps / collab demo / load / README.md

13. GCP: initial startup, acceptance still pending



Target time: 45 seconds

Speaking points

- The user has shown a running service and HTTP 200 on the initial VM. That establishes initial startup, not that the latest features are deployed or that recovery and backups have passed acceptance.
- One Linux VM owns the persistent SQLite data and workspace. Private access uses an SSH tunnel; public HTTPS through a reverse proxy and IP-based hostname was requested and instructions supplied, but no working public URL is recorded.
- Keep the app and Ollama bound to loopback and preserve token authentication. A public URL is a route to the login screen, not permission to use the agents. Linux child cleanup, service-user model access, restore and evaluator access still need explicit checks.

Suggested demonstration: Before the interview, deploy the updated reviewed revision, build and verify the service and chosen access route. For private Windows access use gcloud compute ssh $VmName --project=$ProjectId --zone=$Zone --tunnel-through-iap --ssh-flag="-N" --ssh-flag="-L 14310:127.0.0.1:4310", then open http://127.0.0.1:14310 in two profiles. Use the actual VM zone (us-central1-c for the current VM). Use a public HTTPS URL only after independently verifying it; no endpoint is supplied in this deck.

Fallback: Show the deployment diagram and distinguish user-confirmed initial startup from the pending public access, new-feature and operational checks. The local screenshots remain available.

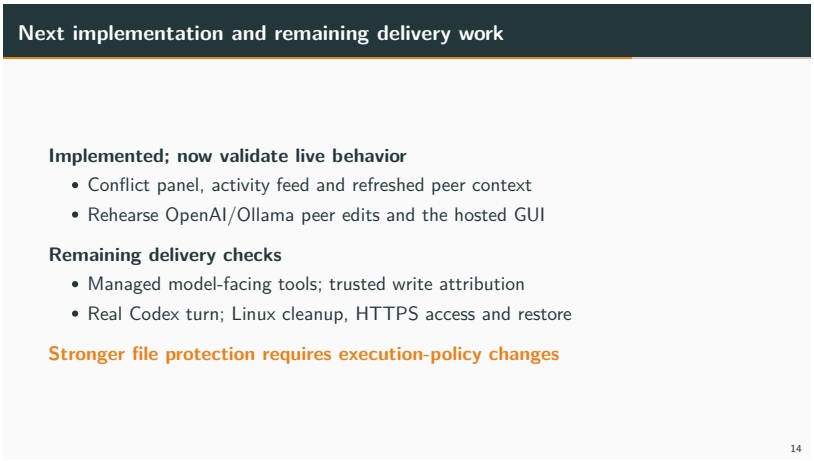
Boundary to state: Do not claim cloud acceptance, production isolation or a successful public endpoint from the initial HTTP 200. VM/storage costs continue without visitors; API use requires credentials. CPU-only Ollama latency must be measured on the chosen VM.

References: Wiki: why the proposed gcp deployment uses one vm

Wiki: llm wiki / interview questions / 08 scaling and operations.md

Source: apps / collab demo / deploy / README.md

14. Next implementation and remaining delivery work



Target time: 40 seconds

Speaking points

- The conflict walkthrough, bounded activity feed and refreshed collaboration guidance are implemented. The next validation is a short real OpenAI/Ollama peer-edit rehearsal plus deployment and recording of the updated GUI.
- The next protection slice is model-facing managed tools bound to trusted request identity, with actual file-effect provenance and a policy for bypass writes. Simply adding more reminders cannot guarantee safe publication.
- Real authenticated Codex turn/continuation, Linux containment and backup restoration remain delivery checks. Worktrees, automatic merge, multi-file transactions, roles/approvals and independent-thread workers are separate extensions with costs.

Suggested demonstration: Return to the implemented conflict/activity panels for questions. If discussing awareness, use a disposable file: Ollama writes, GPT changes it, Ollama rereads and explains the current bytes. State that this live interpretation check is planned until it has actually been run.

Fallback: End on the concrete next slice and its boundaries, then use backup slides only for relevant questions.

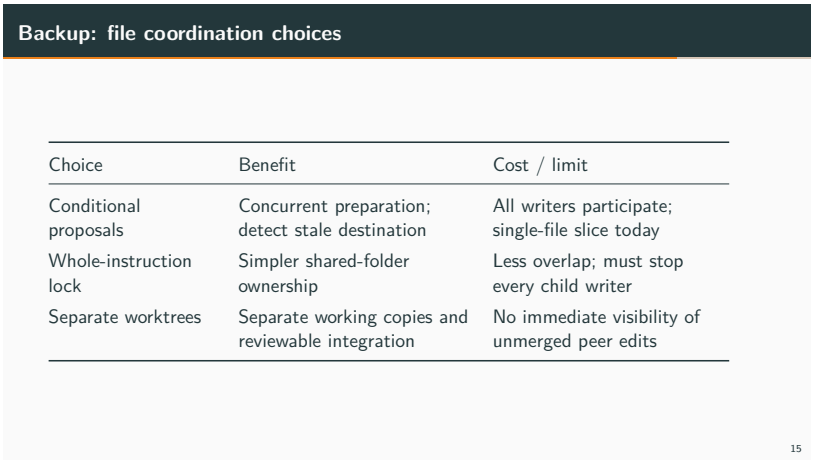
Boundary to state: Local scripted rehearsal needs no key or cloud. Native live checks use the existing OpenAI key and Ollama. External Codex needs separate authentication; GCP demonstrations require deploying the current revision and verifying access. No such calls or cloud changes were made while updating this deck.

References: Wiki: what is tested and what to demonstrate next

Wiki: what would you implement next and what would you defer

Source: current risks and open checks

15. Backup: file coordination choices



Target time: Backup slide

Speaking points

- Optimistic checks allow independent preparation and reject an outdated destination at publication. This protects cooperative writes but does not validate every file used by a plan, and exact text checks do not detect content changing away and back.
- Whole-instruction ownership is simpler for unrestricted harnesses but requires every writer to honor the rule until all child processes stop. A lease expiring is not enough to stop an old shell.
- Separate worktrees create separate working files; branches alone do not. Integration needs review, merge, and testing, and changes the assignment's immediate shared-folder visibility. T3's inspected startup workspace lease ends before the provider turn; it is not a whole-turn lock.

Suggested demonstration: Use example 41 in a separate VM terminal for the implemented conditional-write choice. Discuss worktrees and whole-instruction locking as designs, not hidden modes of the current app.

Fallback: Work through base/current/proposed and ask which trade-off best fits a requirement for unrestricted shell tools.

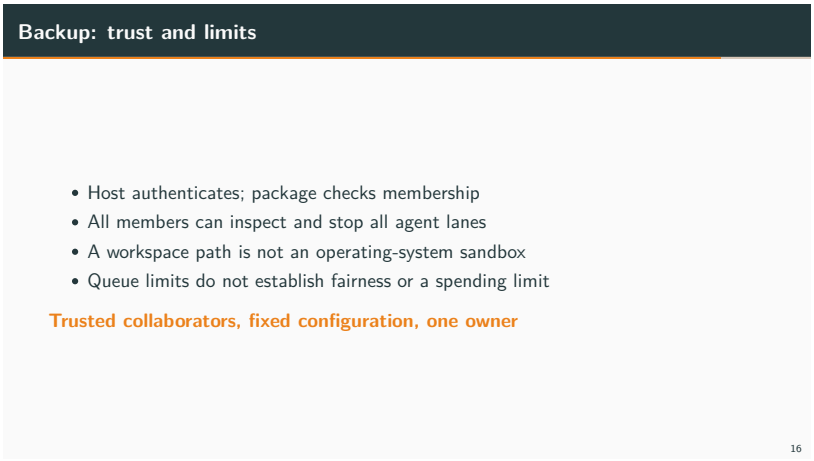
Boundary to state: No automatic Git stash/pop, merge engine, global workspace lock, or multi-file transaction is implemented.

References: Wiki: llm wiki / interview questions / 03 file conflicts.md

Wiki: locks ownership leases and fencing

Source: apps / collab demo / CONFLICT_RESOLUTION_PROPOSAL.md

16. Backup: trust and limits



Target time: Backup slide

Speaking points

- Authentication maps the bearer token to Alice; membership authorization is enforced inside the package. Passing the string alice to the library is not authentication, and a prompt claiming to be an administrator carries no additional authority.
- All thread members intentionally see all lanes and can stop peer work. Fixed membership has no dynamic revocation flow, and the trusted coding process can access credentials available to its service account.
- The default limits include 256 lifetime commands and 16 pending per agent, but one user can consume capacity and one command can call a model many times. A public service needs a different identity, isolation, budget, and retention design.

Suggested demonstration: If asked, show the GUI's shared scope with Alice and Bob. Use existing automated unauthorized-access tests as evidence instead of displaying real tokens or attempting destructive commands.

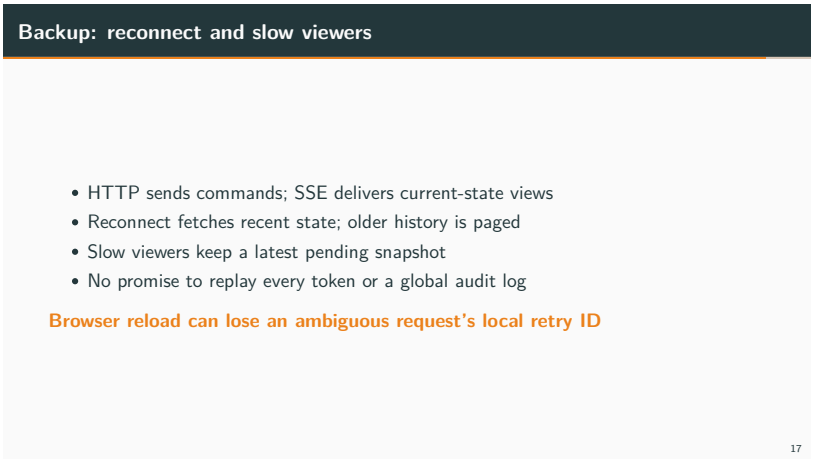
Fallback: Contrast host identity checks with package membership, then explain why neither is an OS sandbox.

Boundary to state: Path/link checks in the read-only viewer are hygiene for trusted users, not confinement against hostile concurrent writers or prompt injection.

References: Wiki: authentication membership and authority

Wiki: llm wiki / interview questions / 06 security and api.md

17. Backup: reconnect and slow viewers



Target time: Backup slide

Speaking points

- The browser gets current-state snapshots over SSE, not a durable stream of every intermediate token. It can page older committed messages, and stable entry IDs avoid duplicate display after reconnect.
- One shared watch observes each lane; a blocked viewer keeps only a latest pending snapshot. Limits constrain viewers, frames, and application buffering, not all kernel buffers or total process memory.
- A browser can forget an ambiguous draft's retry ID on reload even while the server retains accepted work. A future saved client receipt/draft list would need explicit retention and reconciliation, without storing bearer tokens.

Suggested demonstration: Disconnect only Alice's browser connection, let Bob observe accepted work, reconnect Alice and inspect history. Reauthenticate after reload if needed; do not resend the instruction under a new ID just to restore the display.

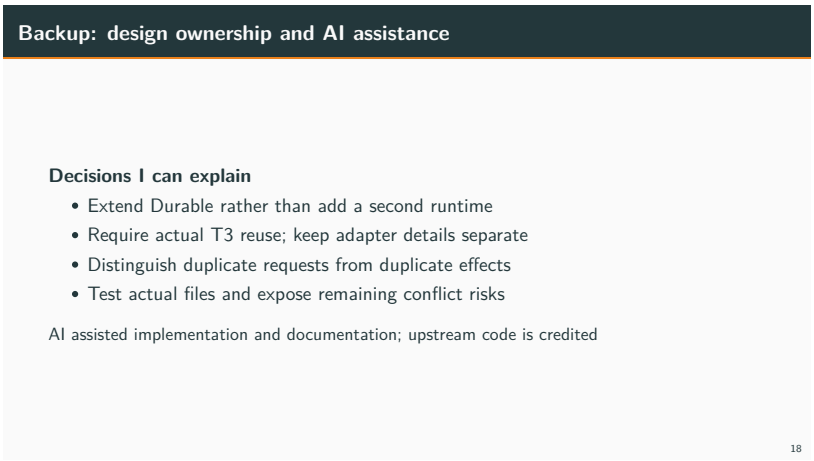
Fallback: Explain that a latest-view stream can omit progress steps while still catching up with completed messages and state.

Boundary to state: The views combine sources and are not an atomic cross-source snapshot or a global audit log. No GUI network-fault toggle or durable replay cursor is provided.

References: Wiki: snapshots are convergent views not an event journal

Wiki: backpressure fan out and coalescing

18. Backup: design ownership and AI assistance



Target time: Backup slide

Speaking points

- I directed the scope toward evolving Durable itself, required real T3 reuse, challenged whether the demo truly showed M:N sharing, and kept the file-conflict limitations visible. AI assistance helped implement, investigate, and document the work.
- I should be able to explain a request, a crash boundary, and a lost-update trace without relying on an assistant's summary. Existing Pi storage/tools/providers and T3 client code remain credited upstream work.
- The original approximately twenty-hour target shaped prioritization, but it is not a measured claim about actual elapsed development time. I would not claim I manually wrote all code or that passing tests substitute for understanding it.

Suggested demonstration: No new app interaction is needed. Answer a concrete follow-up using a source file, wiki explanation, or the controlled example rather than opening a large unprepared code tour.

Fallback: Explain one trade-off you deliberately accepted and what evidence would justify changing it.

Boundary to state: Treat these as preparation points to phrase truthfully in your own words; do not invent a personal development history or an unrecorded result.

References: Wiki: how do you explain ai assisted development

Wiki: how would you explain your own judgment when much of the implementation used ai assistance